

SOLAR ROBO

Android System Architecture, Module Specification & Agentic Development Workflow

Precise implementation contract for developers and AI coding agents

Core principle: create the puzzle piece completely → verify it → place it in the correct position.

```
RAPID LOOP
AI Agent / Developer
↓
Same local Git working tree
↓
Android Studio: Compose Preview / Live Edit
↓
Emulator / Device
↓
Human review → Git commit + push
↓
GitHub Actions: build + tests + lint + architecture
↓
PR / Merge
```

GitHub is the source-of-truth/CI layer, not the primary live-preview transport. The agent and Android Studio use the same local checkout for rapid feedback.

1. Purpose & Rules

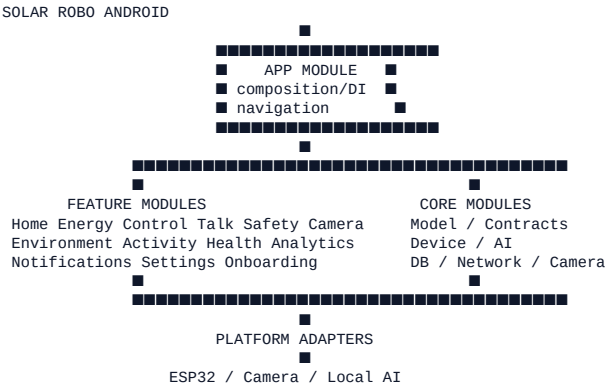
This document locks the Android technology stack, system architecture, exact repository structure, module responsibilities, function I/O, skeleton code, rapid-preview workflow, agentic tooling and continuous integration. A coding model should be able to implement one module without inventing architecture.

- **Native Android:** Kotlin + Jetpack Compose only.
- **Feature isolation:** feature modules never import other feature modules.
- **Contracts:** cross-boundary data uses canonical core contracts.
- **Mock-first:** every feature works without real ESP32/camera/AI/cloud.
- **Safety:** AI can recommend, but physical commands pass a deterministic safety gate.
- **No scope expansion:** implement only the assigned task.
- **Rapid feedback:** Compose Preview and Live Edit for supported UI changes.
- **CI:** every push/PR is automatically verified.
- **Stop condition:** if another module private internals are needed, report a contract gap instead of inventing a workaround.

2. Locked Technology Stack

Area	Technology	Role
Platform	Android	Native target
Language	Kotlin	Application source
UI	Jetpack Compose	Screens/components/previews
Architecture	Clean Architecture + MVVM/UDF	Boundaries/state
DI	Hilt	Dependency injection
Async	Coroutines + Flow/StateFlow	Reactive/background work
Navigation	Navigation Compose	Routes
Persistence	Room + DataStore	History + preferences
Network	OkHttp + Retrofit + WebSocket	REST/realtime where needed
Device	Android BLE + Wi-Fi/WebSocket adapter	Discovery/provisioning/normal operation
Camera	CameraX	Preview/snapshot
Background	WorkManager	Deferred work
AI	Custom AiEngine interface	Model abstraction
Testing	JUnit + AndroidX Test + Compose UI Test	Verification
Build	Gradle Kotlin DSL	Build/dependencies
VCS	Git + GitHub	Source control
CI	GitHub Actions	Build/test/lint/architecture
Rapid preview	Compose @Preview + Live Edit	Fast UI iteration
Agents	Antigravity / Claude / Codex / Android Studio agents	Agentic development
AI experiments	Google AI Studio	Prompt/model experiments

3. System Architecture



Dependency rule

Allowed: app → features + core; feature → core
Forbidden: feature:energy → feature:home; feature:talk → feature:control
Cross-feature: producer → core contract/event boundary → consumer

Runtime path

ESP32 / simulator / camera / AI → adapter → canonical contract → repository/Flow
→ ViewModel → Compose UI → user command → SAFETY GATE → device adapter → ESP32

4. Exact Repository Structure

```
SolarRobo/
  ■■■ settings.gradle.kts
  ■■■ build.gradle.kts
  ■■■ app/
  ■   ■■■ build.gradle.kts
  ■   ■■■ src/main/java/com/solarrobo/app/
  ■       ■■■ SolarRoboApplication.kt
  ■       ■■■ MainActivity.kt
  ■       ■■■ App.kt
  ■       ■■■ navigation/AppNavHost.kt
  ■■■ core/
  ■   ■■■ model/ ■■■ contracts/ ■■■ ui/ ■■■ common/
  ■   ■■■ device/ ■■■ ai/ ■■■ network/ ■■■ database/
  ■   ■■■ storage/ ■■■ camera/ ■■■ simulator/
  ■■■ feature/
  ■   ■■■ onboarding/ ■■■ home/ ■■■ energy/ ■■■ control/
  ■   ■■■ talk/ ■■■ safety/ ■■■ camera/ ■■■ environment/
  ■   ■■■ activity/ ■■■ health/ ■■■ analytics/
  ■   ■■■ notifications/ ■■■ settings/
  ■■■ docs/
  ■   ■■■ SYSTEM_ARCHITECTURE.md
  ■   ■■■ TECH_STACK.md
  ■   ■■■ MODULE_CONTRACTS.md
  ■   ■■■ AGENT_RULES.md
  ■   ■■■ modules/<MODULE>.md
  ■■■ .github/workflows/android-ci.yml
  ■■■ AGENTS.md
  ■■■ README.md
```

Standard feature structure

```
feature/energy/
  ■■■ build.gradle.kts
  ■■■ src/main/java/com/solarrobo/feature/energy/
  ■       ■■■ EnergyModule.kt
  ■       ■■■ data/EnergyDataSource.kt
  ■       ■■■ data/EnergyRepositoryImpl.kt
  ■       ■■■ domain/EnergyRepository.kt
  ■       ■■■ domain/CalculateEnergyMetrics.kt
  ■       ■■■ presentation/EnergyViewModel.kt
  ■       ■■■ presentation/EnergyUiState.kt
  ■       ■■■ presentation/EnergyScreen.kt
  ■       ■■■ components/EnergyFlowCard.kt
  ■       ■■■ components/BatteryCard.kt
  ■       ■■■ components/EnergyHistoryChart.kt
  ■       ■■■ mock/FakeEnergyRepository.kt
src/test/.../EnergyViewModelTest.kt
src/test/.../CalculateEnergyMetricsTest.kt
```

5. Core Contracts & Adapter Skeletons

```
// core/contracts/RoboSnapshot.kt
data class RoboSnapshot(
    val deviceId:String, val name:String, val connected:Boolean,
    val mode:RoboMode, val panelAngleDeg:Float, val targetAngleDeg:Float,
    val generationWatts:Float, val batteryPercent:Float, val timestamp:Long)
enum class RoboMode { NORMAL, OPTIMIZING, CONSERVING, PROTECTING, FAULT, SAFE }

// core/contracts/RoboCommand.kt
sealed interface RoboCommand {
    data class MoveToAngle(val angleDeg:Float):RoboCommand
    data object StopMotion:RoboCommand
    data object SafePosition:RoboCommand
}
data class CommandResult(val accepted:Boolean, val commandId:String, val reason:String?=null)

// core/device/RoboDevice.kt
interface RoboDevice {
    suspend fun connect():Result<Unit>
    suspend fun disconnect()
    suspend fun getSnapshot():RoboSnapshot
    suspend fun sendCommand(command:RoboCommand):CommandResult
    fun observeSnapshot():Flow<RoboSnapshot>
}

// core/ai/AiEngine.kt
data class AiRequest(val message:String, val context:Map<String,Any?>)
data class AiResponse(val text:String, val intent:String?=null, val confidence:Float?=null)
interface AiEngine { suspend fun generate(request:AiRequest):AiResponse }
```

6. Module Master Matrix

Module	Exact task	Inputs	Outputs	Must not
Onboarding	Discover/connect/configure	Discovery + user config	DeviceConfig + event	Movement
Home	First-glance status	Robo/Energy/Safety	HomeUiState	Own sources
Energy	Energy display/history	Snapshots/history	EnergyUiState	Physical control
Control	Manual commands	Robo + user action	RoboCommand/result	Bypass safety
Talk	Conversation/AI	Message/context/AiEngine	Messages + intent	Direct actuation
Safety	Deterministic gate/emergency	Command/env/health	Decision/events	LLM safety
Camera	Preview/snapshot	Camera adapter/action	CameraState/frame	VLM
Environment	Current conditions	EnvironmentSnapshot	EnvironmentUiState	Forecast
Activity	Event history	ActivityEvent/filter	Timeline	Invent events
Health	Diagnostics	DeviceHealth	HealthUiState	Repair
Analytics	Historical metrics	Energy/activity	AnalyticsReport	Control
Notifications	Alert inbox	NotificationEvent	List/read state	Root cause
Settings	Preferences	DataStore/edits	Changed event	Safety limits
Simulator	Fake deterministic world	Scenarios/ticks	Canonical snapshots/events	Real hardware

The module sections below are authoritative. An assigned model must not infer extra work from another module.

7. Onboarding

Discover, connect and configure a Robo without operating it.

Exact files

```
feature/onboarding/
  build.gradle.kts
  src/main/java/com/solarrobo/feature/onboarding/
    OnboardingModule.kt
    domain/OnboardingRepository.kt
    data/OnboardingRepositoryImpl.kt
    presentation/OnboardingViewModel.kt
    presentation/OnboardingUiState.kt
    presentation/OnboardingScreen.kt
    components/OnboardingCard.kt
    mock/FakeOnboardingRepository.kt
src/test/.../OnboardingViewModelTest.kt
```

Module I/O

Dir	Resource	Boundary	Meaning
IN	DeviceCandidate[]	Discovery	Available devices
IN	DeviceConfig	UI	Name/location/identity
OUT	OnboardingResult	ViewModel	Success/failure
OUT	DeviceAdded	Event bus	Completion event

Function-level tasks

Function	Input	Output	Exact task
scanDevices()	none	List<Candidate>	Discover/map errors
connectDevice(id)	String	Result<Unit>	Connect/verify
saveConfig(c)	DeviceConfig	Result<Unit>	Persist config
complete(c)	DeviceConfig	Result	Connect→save→publish

Execution flow

```
scan → select → connect → configure → save → publish → success
```

Must NOT implement

- No movement.
- No optimization.
- No AI.
- No safety implementation.

8. Home

Read-only command center for current Robo, energy, connection and active safety alerts.

Exact files

```
feature/home/
  ■■■ build.gradle.kts
  ■■■ src/main/java/com/solarrobo/feature/home/
    ■■■ HomeModule.kt
    ■■■ domain/HomeRepository.kt
    ■■■ data/HomeRepositoryImpl.kt
    ■■■ presentation/HomeViewModel.kt
    ■■■ presentation/HomeUiState.kt
    ■■■ presentation/HomeScreen.kt
    ■■■ components/HomeCard.kt
    ■■■ mock/FakeHomeRepository.kt
src/test/.../HomeViewModelTest.kt
```

Module I/O

Dir	Resource	Boundary	Meaning
IN	RoboSnapshot	Robo repository	Current state
IN	EnergySnapshot	Energy repository	Current energy
IN	SafetyEvent[]	Safety boundary	Active alerts
OUT	HomeUiState	Compose	Dashboard state
OUT	NavigationIntent	App	Navigation only

Function-level tasks

Function	Input	Output	Exact task
loadHome()	none	HomeUiState	Combine declared inputs
formatRoboStatus(s)	RoboSnapshot	StatusModel	Display mapping
activeAlerts(e)	SafetyEvent[]	AlertItem[]	Filter active alerts

Execution flow

```
collect → map → render → navigation
```

Must NOT implement

- No commands.
- No AI.
- No safety decisions.
- No source ownership.

9. Energy

Display current and historical energy; never control physical hardware.

Exact files

```
feature/energy/
■■■ build.gradle.kts
■■■ src/main/java/com/solarrobo/feature/energy/
    ■■■ EnergyModule.kt
    ■■■ domain/EnergyRepository.kt
    ■■■ data/EnergyRepositoryImpl.kt
    ■■■ presentation/EnergyViewModel.kt
    ■■■ presentation/EnergyUiState.kt
    ■■■ presentation/EnergyScreen.kt
    ■■■ components/EnergyCard.kt
    ■■■ mock/FakeEnergyRepository.kt
src/test/.../EnergyViewModelTest.kt
```

Module I/O

Dir	Resource	Boundary	Meaning
IN	EnergySnapshot	Device/repository	Current measurements
IN	EnergyHistoryPoint[]	Room/repository	History
OUT	EnergyUiState	Compose	Cards/charts
OUT	EnergyRangeRequest	Repository	Requested range

Function-level tasks

Function	Input	Output	Exact task
observeEnergy()	none	Flow<Snapshot>	Expose current state
loadHistory(range)	DateRange	List<Point>	Read selected range
calculateTotal(p)	Point[]	Float	Sum real Wh
buildUiState(s,h)	Snapshot+history	UiState	Combine/format

Execution flow

```
observe → load history → calculate → emit UI state
```

Must NOT implement

- No panel movement.
- No battery-limit changes.
- No grid export actuation.
- No weather forecast.
- No fabricated data.

Skeleton

```
class CalculateEnergyMetrics { operator fun invoke(p:List<EnergyHistoryPoint>)=p.sumOf{it.generatedWh.toDouble()}.toFloat() }
```

10. Control

Manual/explicit panel movement through typed commands and the safety boundary.

Exact files

```
feature/control/
■■■ build.gradle.kts
■■■ src/main/java/com/solarrobo/feature/control/
    ■■■ ControlModule.kt
    ■■■ domain/ControlRepository.kt
    ■■■ data/ControlRepositoryImpl.kt
    ■■■ presentation/ControlViewModel.kt
    ■■■ presentation/ControlUiState.kt
    ■■■ presentation/ControlScreen.kt
    ■■■ components/ControlCard.kt
    ■■■ mock/FakeControlRepository.kt
src/test/.../ControlViewModelTest.kt
```

Module I/O

Dir	Resource	Boundary	Meaning
IN	RoboSnapshot	Device/repository	Angle/mode
IN	Angle/action	User	Requested operation
OUT	RoboCommand	Safety	Typed request
OUT	CommandResult	Device	Result

Function-level tasks

Function	Input	Output	Exact task
validateAngle(a)	Float	Result<Float>	Input validation only
createMoveCommand(a)	Float	RoboCommand	Create command
sendMove(a)	Float	CommandResult	Submit to safety boundary
stopMotion()	none	CommandResult	Stop
safePosition()	none	CommandResult	Safe position

Execution flow

```
user angle → validate → command → safety → device → result
```

Must NOT implement

- No GPIO.
- No ESP32 protocol in UI.
- No AI reasoning.
- No safety bypass.

Skeleton

```
fun createMoveCommand(a:Float):RoboCommand=RoboCommand.MoveToAngle(a)
```

11. Talk

Conversation and AI presentation; inference remains behind AiEngine.

Exact files

```
feature/talk/
  build.gradle.kts
  src/main/java/com/solarrobo/feature/talk/
    TalkModule.kt
    domain/TalkRepository.kt
    data/TalkRepositoryImpl.kt
    presentation/TalkViewModel.kt
    presentation/TalkUiState.kt
    presentation/TalkScreen.kt
    components/TalkCard.kt
    mock/FakeTalkRepository.kt
src/test/.../TalkViewModelTest.kt
```

Module I/O

Dir	Resource	Boundary	Meaning
IN	Message/transcript	UI/voice	User text
IN	Context	Repository	Bounded history
IN	AiEngine	Core	Inference
OUT	ConversationMessage	State	Messages
OUT	AI intent	Event/contract	Optional intent

Function-level tasks

Function	Input	Output	Exact task
buildRequest(m,c)	String+context	AiRequest	Build bounded request
sendMessage(m)	String	Message	Invoke engine/update
saveSummary(s)	Summary	Result	Persist important summary

Execution flow

```
message → request → AiEngine → response → store/display → optional intent
```

Must NOT implement

- No direct movement.
- No safety override.
- No model SDK in UI.
- No unlimited raw history.

Skeleton

```
class TalkViewModel(private val ai:AiEngine):ViewModel(){ fun send(m:String)=viewModelScope.launch { val r=ai.generate(AiRequest(m,emptyMap()));
```

12. Safety & Emergency

Deterministic physical-command gate, emergency stop, safe position and incidents.

Exact files

```
feature/safety/
■■■ build.gradle.kts
■■■ src/main/java/com/solarrobo/feature/safety/
    ■■■ SafetyModule.kt
    ■■■ domain/SafetyRepository.kt
    ■■■ data/SafetyRepositoryImpl.kt
    ■■■ presentation/SafetyViewModel.kt
    ■■■ presentation/SafetyUiState.kt
    ■■■ presentation/SafetyScreen.kt
    ■■■ components/SafetyCard.kt
    ■■■ mock/FakeSafetyRepository.kt
src/test/.../SafetyViewModelTest.kt
```

Module I/O

Dir	Resource	Boundary	Meaning
IN	RoboCommand	Control boundary	Requested action
IN	EnvironmentSnapshot	Environment	Conditions
IN	DeviceHealth	Health/device	Fault context
OUT	SafetyDecision	Control/device	ALLOW/BLOCK/MODIFY
OUT	SafetyEvent	Event/storage	Incident
OUT	Safe command	Device	STOP/SAFE_POSITION

Function-level tasks

Function	Input	Output	Exact task
evaluate(c,ctx)	Command+Context	SafetyDecision	Apply deterministic rules
emergencyStop(r)	String	CommandResult	Stop + event
safePosition(r)	String	CommandResult	Safe + event
acknowledge(id)	String	Unit	Persist acknowledgement

Execution flow

```
command → evaluate → block/modify/allow → device → observe → update
```

Must NOT implement

- No LLM safety.
- No silent threshold changes.
- No duplicate telemetry owner.

Skeleton

```
sealed interface SafetyDecision { data object Allow:SafetyDecision; data class Block(val reason:String):SafetyDecision; data class Modify(val cor
```

13. Camera

CameraX preview/connection/snapshot; vision reasoning is separate.

Exact files

```
feature/camera/
■■■ build.gradle.kts
■■■ src/main/java/com/solarrobo/feature/camera/
    ■■■ CameraModule.kt
    ■■■ domain/CameraRepository.kt
    ■■■ data/CameraRepositoryImpl.kt
    ■■■ presentation/CameraViewModel.kt
    ■■■ presentation/CameraUiState.kt
    ■■■ presentation/CameraScreen.kt
    ■■■ components/CameraCard.kt
    ■■■ mock/FakeCameraRepository.kt
src/test/.../CameraViewModelTest.kt
```

Module I/O

Dir	Resource	Boundary	Meaning
IN	CameraState/frame	Camera adapter	Connection/frames
IN	Camera action	User	Start/stop/snapshot
OUT	CameraUiState	Compose	Preview/error
OUT	Frame	Vision boundary	Requested snapshot

Function-level tasks

Function	Input	Output	Exact task
start()	none	Result<Unit>	Start stream
stop()	none	Unit	Stop
capture()	none	Result<Frame>	Capture one
observeState()	none	Flow<State>	Expose state

Execution flow

```
permission → start → preview → snapshot → frame
```

Must NOT implement

- No face recognition.
- No VLM.
- No safety decision from image.
- No continuous video persistence.

14. Environment

Present current local environmental measurements.

Exact files

```
feature/environment/
■■■ build.gradle.kts
■■■ src/main/java/com/solarrobo/feature/environment/
    ■■■ EnvironmentModule.kt
    ■■■ domain/EnvironmentRepository.kt
    ■■■ data/EnvironmentRepositoryImpl.kt
    ■■■ presentation/EnvironmentViewModel.kt
    ■■■ presentation/EnvironmentUiState.kt
    ■■■ presentation/EnvironmentScreen.kt
    ■■■ components/EnvironmentCard.kt
    ■■■ mock/FakeEnvironmentRepository.kt
src/test/.../EnvironmentViewModelTest.kt
```

Module I/O

Dir	Resource	Boundary	Meaning
IN	EnvironmentSnapshot	Device/simulator	Temperature/humidity/light/wind/rain/panel temp
OUT	EnvironmentUiState	Compose	Formatted conditions
OUT	ConditionEvent	Event bus	Significant change

Function-level tasks

Function	Input	Output	Exact task
observeEnvironment()	none	Flow<Snapshot>	Observe
toUiState(s)	Snapshot	UiState	Format
detectChange(p,c)	Snapshots	Boolean	Configured change detection

Execution flow

```
snapshot → validate → format → UI → optional event
```

Must NOT implement

- No forecast engine.
- No external weather API unless assigned.
- No panel control.
- No safety thresholds in UI.

15. Activity / Robo Memory

Persist/display important events and summaries; do not store every raw telemetry sample.

Exact files

```
feature/activity/
  build.gradle.kts
  src/main/java/com/solarrobo/feature/activity/
    ActivityModule.kt
    domain/ActivityRepository.kt
    data/ActivityRepositoryImpl.kt
    presentation/ActivityViewModel.kt
    presentation/ActivityUiState.kt
    presentation/ActivityScreen.kt
    components/ActivityCard.kt
    mock/FakeActivityRepository.kt
src/test/.../ActivityViewModelTest.kt
```

Module I/O

Dir	Resource	Boundary	Meaning
IN	ActivityEvent	Event bus	Important event
IN	ActivityFilter	User	Filter
OUT	ActivityEvent[]	Compose	Timeline

Function-level tasks

Function	Input	Output	Exact task
append(e)	ActivityEvent	Result	Persist
query(f)	Filter	List<Event>	Newest-first
summarize(e)	Event	String	Display summary

Execution flow

```
event → normalize → persist → query → filter → render
```

Must NOT implement

- No invented events.
- No telemetry firehose.
- No analytics.

16. Health & Diagnostics

Present standardized component/device health.

Exact files

```
feature/health/
■■■ build.gradle.kts
■■■ src/main/java/com/solarrobo/feature/health/
    ■■■ HealthModule.kt
    ■■■ domain/HealthRepository.kt
    ■■■ data/HealthRepositoryImpl.kt
    ■■■ presentation/HealthViewModel.kt
    ■■■ presentation/HealthUiState.kt
    ■■■ presentation/HealthScreen.kt
    ■■■ components/HealthCard.kt
    ■■■ mock/FakeHealthRepository.kt
src/test/.../HealthViewModelTest.kt
```

Module I/O

Dir	Resource	Boundary	Meaning
IN	DeviceHealth	Device	Motor/battery/sensor/controller/camera/network
OUT	HealthUiState	Compose	Checklist/details
OUT	HealthEvent	Event bus	Important change

Function-level tasks

Function	Input	Output	Exact task
loadHealth()	none	DeviceHealth	Read
toUiState(h)	Health	UiState	Map
criticalIssues(h)	Health	Issue[]	Identify declared criticals

Execution flow

```
health → map → render → meaningful event
```

Must NOT implement

- No automatic repair.
- No motor parameter changes.
- No new telemetry protocol.

17. Analytics & Performance

Compute read-only historical metrics from actual energy/activity data.

Exact files

```
feature/analytics/
■■■ build.gradle.kts
■■■ src/main/java/com/solarrobo/feature/analytics/
    ■■■ AnalyticsModule.kt
    ■■■ domain/AnalyticsRepository.kt
    ■■■ data/AnalyticsRepositoryImpl.kt
    ■■■ presentation/AnalyticsViewModel.kt
    ■■■ presentation/AnalyticsUiState.kt
    ■■■ presentation/AnalyticsScreen.kt
    ■■■ components/AnalyticsCard.kt
    ■■■ mock/FakeAnalyticsRepository.kt
src/test/.../AnalyticsViewModelTest.kt
```

Module I/O

Dir	Resource	Boundary	Meaning
IN	EnergyHistoryPoint[]	Repository	Measurements
IN	ActivityEvent[]	Activity	Operational history
OUT	AnalyticsReport	ViewModel	Metrics/charts

Function-level tasks

Function	Input	Output	Exact task
calculateEnergyTotal(p)	Points	Float	Sum actual generation
calculateBatteryUtilization(d)	Energy data	Float	Apply documented formula
buildReport(e,a)	Datasets	Report	Build report

Execution flow

```
load → validate → calculate → mark missing → render
```

Must NOT implement

- No fabricated comparisons.
- No device control.
- No raw-history mutation.

18. Notifications

Consume standardized notification events, deduplicate and manage read state.

Exact files

```
feature/notifications/
■■■ build.gradle.kts
■■■ src/main/java/com/solarrobo/feature/notifications/
    ■■■ NotificationsModule.kt
    ■■■ domain/NotificationsRepository.kt
    ■■■ data/NotificationsRepositoryImpl.kt
    ■■■ presentation/NotificationsViewModel.kt
    ■■■ presentation/NotificationsUiState.kt
    ■■■ presentation/NotificationsScreen.kt
    ■■■ components/NotificationsCard.kt
    ■■■ mock/FakeNotificationsRepository.kt
src/test/.../NotificationsViewModelTest.kt
```

Module I/O

Dir	Resource	Boundary	Meaning
IN	NotificationEvent	Event bus	New alert
IN	Read action	User	ID
OUT	Notification[]	Compose	Inbox
OUT	Read state	Storage/event	Updated state

Function-level tasks

Function	Input	Output	Exact task
ingest(e)	Event	Unit	Normalize/deduplicate/store
markRead(id)	String	Unit	Persist read
unreadCount()	none	Int	Badge count

Execution flow

```
event → deduplicate → store → badge → read
```

Must NOT implement

- No root-cause logic.
- No spam.
- No safety decisions.

19. Settings

Manage user/application preferences; safety-critical limits remain outside editable settings.

Exact files

```
feature/settings/
■■■ build.gradle.kts
■■■ src/main/java/com/solarrobo/feature/settings/
    ■■■ SettingsModule.kt
    ■■■ domain/SettingsRepository.kt
    ■■■ data/SettingsRepositoryImpl.kt
    ■■■ presentation/SettingsViewModel.kt
    ■■■ presentation/SettingsUiState.kt
    ■■■ presentation/SettingsScreen.kt
    ■■■ components/SettingsCard.kt
    ■■■ mock/FakeSettingsRepository.kt
src/test/.../SettingsViewModelTest.kt
```

Module I/O

Dir	Resource	Boundary	Meaning
IN	Stored settings	DataStore	Preferences
IN	SettingsPatch	User	Edits
OUT	SettingsChangedEvent	Event bus	Validated change
OUT	Stored settings	DataStore	Saved config

Function-level tasks

Function	Input	Output	Exact task
load()	none	Settings	Read defaults/stored
update(p)	Patch	Settings	Validate + persist
reset()	none	Settings	Restore defaults

Execution flow

```
load → merge → validate → persist → publish
```

Must NOT implement

- No hidden safety thresholds.
- No direct device calls.
- No authentication.

20. Developer Simulator

Deterministic fake Robo/sensor/fault world using production contracts.

Exact files

```
feature/simulator/
■■■ build.gradle.kts
■■■ src/main/java/com/solarrobo/feature/simulator/
    ■■■ SimulatorModule.kt
    ■■■ domain/SimulatorRepository.kt
    ■■■ data/SimulatorRepositoryImpl.kt
    ■■■ presentation/SimulatorViewModel.kt
    ■■■ presentation/SimulatorUiState.kt
    ■■■ presentation/SimulatorScreen.kt
    ■■■ components/SimulatorCard.kt
    ■■■ mock/FakeSimulatorRepository.kt
src/test/.../SimulatorViewModelTest.kt
```

Module I/O

Dir	Resource	Boundary	Meaning
IN	ScenarioConfig	Developer UI	Sun/wind/rain/battery/fault
IN	tick(dt)	Engine	Deterministic time
OUT	RoboSnapshot	Core	Fake Robo
OUT	EnergySnapshot	Core	Fake energy
OUT	EnvironmentSnapshot	Core	Fake environment
OUT	SafetyEvent	Event bus	Injected fault

Function-level tasks

Function	Input	Output	Exact task
applyScenario(s)	Scenario	Unit	Set state
tick(dt)	Long	State	Advance
buildSnapshots(s)	State	Snapshots	Canonical outputs
injectFault(t)	FaultType	SafetyEvent	Controlled fault

Execution flow

```
select → initialize → tick → snapshots → publish → observe
```

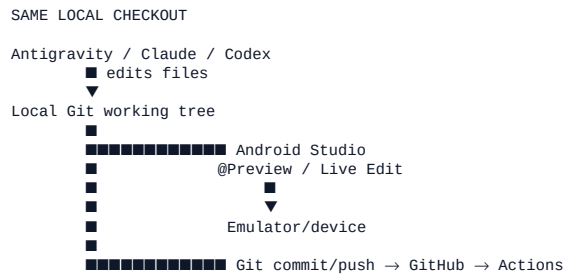
Must NOT implement

- Never connect real hardware.
- Never replace production adapter.
- No hidden randomness.
- No production threshold changes.

Skeleton

```
enum class Scenario { SUNNY_NORMAL, LOW_LIGHT, HIGH_WIND, RAIN, MOTOR_JAM, BATTERY_LOW, DEVICE_OFFLINE }
```

21. Rapid Prototyping & Continuous Preview



How to actually use the tools

- Clone GitHub once; open that exact folder in Android Studio.
- Open the same folder in Antigravity/Claude/Codex, or connect an agent through Android Studio when supported.
- Run emulator/app once.
- Use Compose @Preview for components and Live Edit for supported UI edits.
- Agent changes files in the same working tree; Android Studio sees them.
- Structural changes/new dependencies require normal Gradle build/run.
- Use Developer Simulator to test hardware-like states without hardware.
- Review the diff before commit; push only verified work.

Do not use GitHub as the per-edit preview transport. The live loop is local file change → Android Studio Preview/Live Edit. GitHub is the collaboration/CI loop.

Layer	Tool	Trigger	Result
Component	@Preview	Composable edit	Fast component view
Rapid UI	Live Edit	Supported Compose edit	Running emulator update
Feature	Gradle + tests	Task complete	Compiled/tested feature
CI	GitHub Actions	Push/PR	Shared quality gate

22. Agentic Toolchain — Antigravity, Claude, Codex, Android Studio & AI Studio

Tool	Use	Boundary
Antigravity	Bounded module implementation, commands/tests, iteration	Must obey AGENTS.md/spec
Claude Agent	Complex implementation/debug/refactor/review	Cannot silently change contracts
Codex	Implementation/tests/repository fixes	Cannot bypass boundaries
Android Studio	Compose Preview/Live Edit, Gradle, emulator, diagnostics	Native runtime/IDE
Google AI Studio	Prompt/model experiments, structured-output prototyping	Not Android build system
GitHub	Branches, PRs, review, history	Not live preview
GitHub Actions	Build/test/lint/architecture checks	Not UI preview
Emulator	Runtime validation	Not final hardware validation

Agent loop

```
Human selects exact module
→ branch
→ agent reads AGENTS.md + module spec + contracts
→ plan
→ implement ONLY assigned scope
→ build/tests
→ Android Studio Preview/emulator
→ human diff review
→ commit/push
→ GitHub Actions
→ green PR → merge
```

Android Studio agent integration

Use Android Studio native agent capabilities where appropriate. Where the installed release/channel supports Bring Your Own Agent, connected agents can accelerate implementation. The repository must remain buildable without that preview capability.

Google AI Studio

Use it for AI/model experiments and prompt/structured-output design. Production Android code consumes AiEngine; model-specific code stays behind that boundary.

23. Continuous Integration — GitHub Actions

```
.github/workflows/android-ci.yml

name: Android CI
on:
  push:
    branches: ["**"]
  pull_request:

jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "17"
      - run: ./gradlew assembleDebug
      - run: ./gradlew test
      - run: ./gradlew lint
      - run: ./gradlew architectureCheck
```

CI gates

- Compilation passes.
- Unit tests pass.
- Lint passes.
- Architecture check fails on forbidden feature-to-feature imports.
- Required tests cannot be silently skipped.
- Later: selected emulator/instrumented tests and APK artifacts can be added.

24. AGENTS.md — Exact Rules for Coding Models

```
# AGENTS.md

PROJECT
- Native Android. Kotlin + Jetpack Compose.

ARCHITECTURE
- app composes features.
- features never import other features.
- features depend on core interfaces/contracts.
- device/AI/storage access uses adapters.

SCOPE
- Implement ONLY the assigned task.
- Do not add useful-looking extras.
- If blocked, STOP and report it.

UI
- Compose only; add @Preview.
- Add loading/error/offline/empty states where relevant.
- No hardware/model SDK calls in UI.

DEVICE / AI / SAFETY
- Use RoboDevice/RoboCommand.
- Use AiEngine.
- AI output cannot directly actuate hardware.
- Physical commands pass the safety gate.

VERIFY
- Run tests and Gradle build.
- Check feature dependency boundaries.
- Verify required files.
- Report files, functions, inputs, outputs, tests.

STOP CONDITION
If another module's private internals are required, STOP and report the contract mismatch.
```

Exact module assignment prompt

```
TASK: Implement ONLY <MODULE>.

READ: AGENTS.md, SYSTEM_ARCHITECTURE.md, module spec, relevant contracts/interfaces.

DO: create specified files; implement specified functions; exact I/O; mocks; previews; tests; build; boundary check.

DO NOT: redesign architecture; add features; silently modify contracts; import another feature; access hardware/AI SDKs from UI; bypass safety.

IF BLOCKED: STOP and report missing contract/dependency.

FINAL REPORT: Files / Functions / Inputs / Outputs / Tests / Build / Scope additions.
```


25. Definition of Done

Check	Pass condition
Files	Specified files exist; extras justified
Scope	No unapproved behavior
I/O	All external inputs/outputs documented
Functions	Explicit I/O + one responsibility
UI	Compose screen/components + previews
State	Local ViewModel/StateFlow
Mock	Runs without real hardware/AI/cloud
Tests	Success/failure/boundary cases
Architecture	No feature-to-feature imports
Build	Local Gradle passes
CI	GitHub Actions passes
Preview	Visible in Preview/emulator
Docs	Implementation matches module spec

26. Build Order

Stage	Work	Exit
1	Android project + stack	App launches
2	Core contracts	Canonical types compile
3	App shell/design system	Navigation works
4	Simulator/mocks	Fake Robo world works
5	Onboarding	Fake Robo added
6	Home + Energy	Dashboard consumes contracts
7	Control + Safety	Commands gated
8	Talk	Mock AI works
9	Camera + Environment	Independent UI works
10	Activity + Notifications + Health	Events/diagnostics visible
11	Analytics + Settings	History/preferences work
12	Real adapters	Mocks → ESP32/camera/local AI
13	Physical validation	Controlled real-Robo tests

27. Final Development Model

```
HUMAN
↓ exact module specification
AI AGENT
↓ same local repository
ANDROID STUDIO
  ■■ @Preview
  ■■ Live Edit
  ■■ Gradle
  ■■ Emulator
↓
Developer Simulator
↓ human validation + diff review
↓ Git commit/push
GITHUB ACTIONS
  ■■ Build
  ■■ Unit tests
  ■■ Lint
  ■■ Architecture checks
↓
PR green → merge → next puzzle piece
```

Final principle: We build verified Android feature pieces from the beginning. The agent receives an exact contract; Android Studio gives rapid feedback; the simulator removes hardware dependency; GitHub Actions protects the shared branch.

END OF SPECIFICATION